

El lenguaje Python



Python fue creado por Guido van Rossum a finales de los años 80 en los Países Bajos. Van Rossum comenzó a trabajar en Python como un proyecto paralelo mientras estaba en el Instituto de Investigación Nacional de Matemáticas y Ciencias de la Computación (CWI). Python fue concebido como un sucesor del lenguaje de programación ABC, que Van Rossum había trabajado anteriormente.

Origen e Historia

Cuando comenzó a implementar Python, Guido van Rossum también estaba leyendo los guiones publicados de "Monty Python's Flying Circus", una serie de comedia de la BBC de la década de 1970. Van Rossum pensó que necesitaba un nombre que fuera corto, único y ligeramente misterioso, por lo que decidió llamar al lenguaje Python.



Guido van Rossum

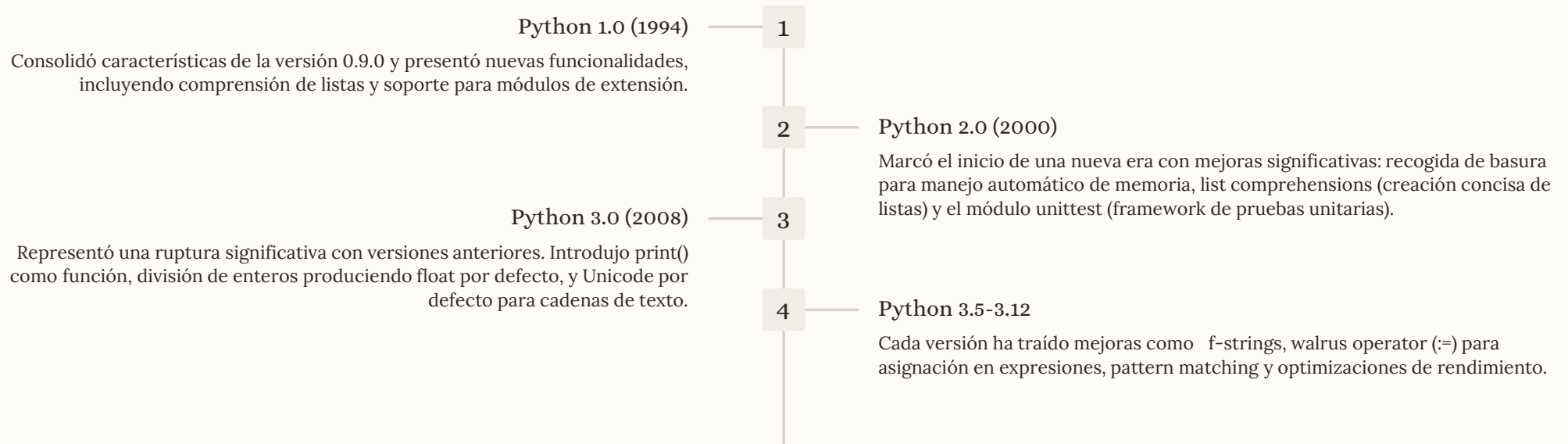
A partir de su publicación, el proyecto ya era de dominio público, continuó como proyecto de código abierto y siguió creciendo. Y esta es una inercia que se mantiene y podemos seguir desde su página oficial: www.python.org.

La primera versión de Python, conocida como Python 0.9.0, fue lanzada en febrero de 1991. Esta versión incluía muchas características que aún están presentes en el lenguaje, como:

- Sintaxis clara y legible.
- Tipos de datos integrados como listas, diccionarios y cadenas.
- Manejo de excepciones.
- Funciones y módulos.

Python 0.9.0 fue un punto de partida crucial, ya que estableció muchas de las bases del lenguaje tal como lo conocemos hoy.

Evolución de Python



Hoy en día, Python es utilizado ampliamente en una gran variedad de campos, incluyendo desarrollo web, análisis de datos, inteligencia artificial, automatización y más.

La comunidad de Python ha crecido significativamente, y el lenguaje ha sido adoptado por empresas de todo el mundo debido a su versatilidad y facilidad de uso.

La Python Software Foundation (PSF), establecida en 2001, juega un papel crucial en la gestión del desarrollo del lenguaje y en la organización de eventos comunitarios como PyCon.

Python ha tenido un impacto profundo en la industria del software y en la educación en ciencias de la computación. Su diseño simple y la gran cantidad de bibliotecas y herramientas disponibles han facilitado el desarrollo de aplicaciones complejas y la investigación en diversas disciplinas.

El futuro de Python parece prometedor, con la **comunidad activa en la evolución del lenguaje** y la mejora continua de sus capacidades. Las próximas versiones seguirán introduciendo nuevas características y mejoras, consolidando a Python como uno de los lenguajes de programación más importantes y ampliamente utilizados en la industria.

Características principales

Lenguaje de alto nivel

Es más cercano al lenguaje humano que al lenguaje máquina, lo que facilita su comprensión y aprendizaje.



Python es:

fácil de aprender,
fácil de enseñar,
fácil de usar,
fácil de entender,
fácil de obtener.

Interpretado

El código se ejecuta línea por línea sin necesidad de ser compilado previamente, lo que agiliza el desarrollo. El intérprete de Python genera archivos intermedios conocidos como bytecode, de manera similar a como lo hace Java, pero de forma implícita (este bytecode no se almacena en archivos .class como en Java, sino que se guarda en archivos .pyc dentro del directorio `__pycache__`). El bytecode es luego ejecutado por la máquina virtual de Python (Python Virtual Machine, PVM)). Esto posibilita tener código muy portable y con capacidad de ejecutarse a mayor velocidad que la de otros lenguajes interpretados.

Tipado dinámico

El tipo de las variables se determina en tiempo de ejecución, no es necesario declararlo explícitamente.

Multiplataforma

Funciona en diferentes sistemas operativos como Windows, Linux y macOS.

Sintaxis clara y legible

Diseñada para ser fácil de leer y comprender, lo que ayuda a reducir errores y facilita la colaboración entre desarrolladores.

Gran variedad de bibliotecas

Facilitan la realización de tareas específicas, como el análisis de datos, el aprendizaje automático, el desarrollo web y más.

Código abierto

Su código fuente está disponible públicamente y puede ser modificado y distribuido libremente.

Multiparadigma

Python es, ante todo, un lenguaje de programación orientado a objetos, pero permite otros paradigmas como la programación procedimental o la programación funcional.

Proyecto de código abierto

Es un proyecto libre, con una comunidad de desarrolladores inmensa. Cualquiera puede sumarse a este proyecto abierto, proponer características o introducir parches para solucionar problemas. Esta naturaleza abierta y su comunidad posibilitan que el lenguaje siga creciendo, integrando nuevas funcionalidades y mejorando cada día.

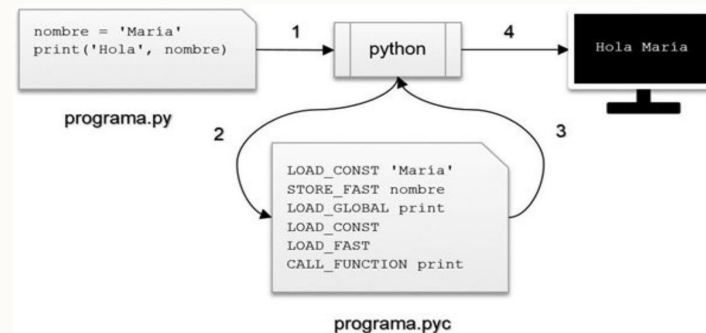
Fácil de aprender

Su sintaxis sencilla y su gran comunidad de usuarios hacen de Python un lenguaje accesible para principiantes y profesionales

El precio de la interpretación

Para ejecutar un programa en Python es necesario un «intérprete», un programa más en nuestro sistema, que, precisamente, se llama python. Cuando ejecutamos esta orden en una terminal, vemos que aparece el símbolo «>>>». Ese es el intérprete de Python esperando recibir instrucciones que, conforme las vayamos introduciendo, serán ejecutadas. Pero cuando escribimos todo nuestro código en un archivo (al que solemos añadir la extensión «.py» para identificar el tipo de contenido como un programa de Python), el intérprete funciona al modo de lenguajes como Java: primero traduce el contenido a un código intermedio o «bytecode», como ya se dijo, y luego el intérprete ejecuta ese código intermedio. Dicho código, si bien está más optimizado, no está compilado en el lenguaje que entiende directamente la máquina, por lo cual debe ser también interpretado.

Comprenderemos mejor este proceso con un ejemplo. Esto es lo que ocurre cuando pedimos a Python que ejecute un programa simple contenido en un archivo al que hemos llamado programa.py:



1

Lectura del código

El intérprete de Python lee el contenido del archivo programa.py.

2

Generación de bytecode

El intérprete genera el bytecode equivalente, o lo que es lo mismo, traduce el lenguaje Python a un lenguaje «intermedio».

3

Preparación para ejecución

Python toma dicho lenguaje intermedio y se prepara para su ejecución. Aquí, el bytecode se pasa al ejecutor de Python.

4

Ejecución

El ejecutor de Python ejecuta las instrucciones contenidas en el bytecode, es decir, el ejecutor le va diciendo a nuestro ordenador, en un lenguaje que sí entiende, qué tiene que hacer.

Todo esto ocurre sin que apenas nos demos cuenta, por lo que para nosotros la sensación es la de ejecutar directamente nuestro programa de Python.

En los programas compilados, el compilador toma el código fuente (código escrito por el programador) y genera un código directamente ejecutable por el sistema, es decir, una vez realizado el proceso de compilación, la máquina puede ejecutar el programa sin necesidad de intermediarios, ya que dicho código fuente se ha traducido (compilado) a código máquina. En el caso de Python, aunque ya esté traducido el código fuente a bytecode, es necesario interpretar dicho bytecode siempre, por lo que cada ejecución implica un proceso de traducción. Esto, de forma inevitable, ralentiza la realización de las tareas por parte de la máquina. En cambio, tiene la ventaja de que el programador no precisa pasar por un proceso de compilación que, además, es diferente según el tipo de ordenador y sistema operativo disponible. Así, conseguimos acelerar el desarrollo y garantizamos la portabilidad^{1,2,3}

Dónde nada Python como pez en el agua



Utilidades del sistema

Generalmente usadas desde el terminal para el procesamiento de archivos, acceso a la red, pruebas de software y un largo etcétera.



Interfaces gráficas de usuario

Para construir aplicaciones de escritorio.

Lenguaje base para el desarrollo de scripts

pequeños programas que añaden funcionalidades a las aplicaciones, como puede ser un efecto en un gráfico 3D o un filtro nuevo para una imagen.



Cálculo numérico y científico

Cuenta con potentes bibliotecas matemáticas, hoy en día muy optimizadas, que proporcionan herramientas potentes de cálculo numérico, estadística y álgebra lineal.



Inteligencia Artificial

Es una de las soluciones más extendidas para construir máquinas pensantes. Técnicas actuales de «aprendizaje automático», entre las que se incluyen las redes neuronales artificiales y profundas.

Educación

Gracias a su sencillez en el lenguaje e inmediatez de ejecución, es una buena forma de iniciarse en la programación.



Aplicaciones web

Dispone de bibliotecas para construir HTML, CSS, JSON o enviar correo electrónico, así como frameworks que nos facilitan el desarrollo de aplicaciones web a más nivel, como Django o Flask, por mencionar solo algunos.



Acceso a bases de datos

Gran conectividad a bases de datos para almacenar información de manera persistente y manipularla en distintos gestores como MySQL o PostgreSQL.

Desarrollo de software

construcción de prototipos o pruebas de concepto en el desarrollo de soluciones informáticas.



Videojuegos

Bibliotecas como Pygame, para crear juegos 2D, y Pyglet, para juegos 3D, nos acercan al mundo del desarrollo de videojuegos de una manera sencilla.



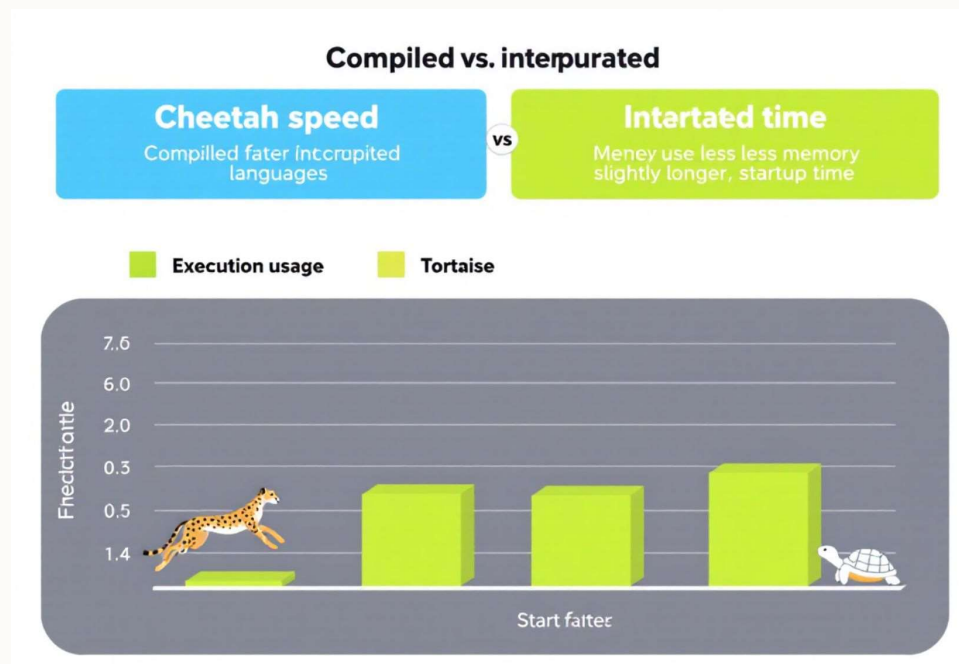
Prototipado hardware

Algunas de las herramientas de enseñanza para la construcción de soluciones hardware más conocidas son Raspberry Pi y Arduino.

Esta lista, aunque amplia, no es completa. Python está presente también en ámbitos como la producción de películas animadas, generación de efectos especiales, programas para el manejo de instalaciones industriales, domótica, el análisis de genes, la búsqueda de vida inteligente en otros planetas o los robots en misiones espaciales. Python es, ante todo, un gran director de orquesta, pues su capacidad de interaccionar con otros programas escritos en C o Java, o de invocar aplicaciones externas, hacen de él una solución muy conveniente para ser el pegamento que lo integre todo. Corporaciones como Google y Facebook optan por Python en muchas de sus soluciones y proyectos. Por algo será, ¿no?

Dónde Python se siente más incómodo

Su gran virtud es su gran lastre. El precio de ser un lenguaje interpretado es demasiado alto en algunas ocasiones. Los ordenadores cada vez son más rápidos y con más memoria, pero en muchas ocasiones las soluciones software necesitan reducir al máximo los recursos que demandan las aplicaciones. Un programa en Python frente a un programa en C, por ejemplo, es más lento (pues requiere ese proceso de interpretación) además de obligar a tener instalado Python y las bibliotecas que el programa necesite. Un programa compilado requiere menos espacio, puede estar más optimizado y es más rápido en su ejecución. Esto suele ser determinante en muchas soluciones industriales.

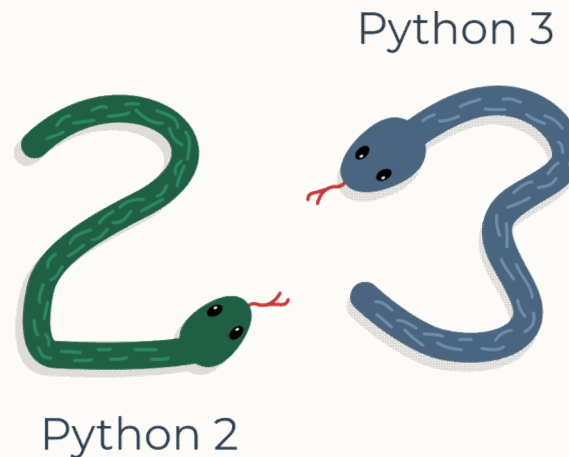


Otros factores también influyen a la hora de optar por un lenguaje para un proyecto de desarrollo: la gran versatilidad de Python no siempre es deseable en equipos de trabajo muy amplios, donde Java y sus fuertes restricciones en tipos de datos y diseño de clases (sobre todo lo referido a la «encapsulación» del código), obliga a que desarrolladores heterogéneos, con estilos de programación muy diferentes, trabajen en proyectos conjuntos.

Python no puede competir allí donde cierto lenguaje es imperativo. Algunos productos obligan a utilizar un determinado lenguaje, por lo que nos vemos obligados a adaptarnos a tales restricciones. Por ejemplo, algunas máquinas industriales solo ofrecen una API o un lenguaje de script determinado. También, en lo relativo a programar aplicaciones para la web, una solución frontend, es decir, una aplicación que se ejecuta en nuestro navegador web, debe utilizar el lenguaje estándar: JavaScript. Existen proyectos que nos permiten programar en lenguaje Python y traducir dicho lenguaje a otro (como C, Java o Javascript). Sin embargo, creemos que un buen programador debe saber elegir la mejor herramienta en cada caso, en lugar de poner en práctica el dicho de «para quien tiene un martillo, todo son clavos».

API son las siglas de Application Programming Interface, es decir, la «interfaz de programación de aplicaciones». Es el conjunto de instrucciones que ofrece una plataforma o servicio para que el desarrollador cree soluciones para dicha plataforma, como puede ser un programa de diseño 3D, un drone o un gestor de bases de datos.

Python 3 o Python 2



Python 3.x no tiene por qué mantener «compatibilidad hacia atrás». Debemos decidir escribir para Python 2 o para Python 3 al escribir nuestros programas, pues usan intérpretes diferentes. En este curso vamos a optar por Python 3 pues, prácticamente todas las bibliotecas más potentes que existían en Python 2 ya tienen su versión para Python 3.

Python ha tenido un impacto profundo en la industria del software y en la educación en ciencias de la computación. Su diseño simple y la gran cantidad de bibliotecas y herramientas disponibles han facilitado el desarrollo de aplicaciones complejas y la investigación en diversas disciplinas.

El futuro de Python parece prometedor, con la comunidad activa en la evolución del lenguaje y la mejora continua de sus capacidades. Las próximas versiones seguirán introduciendo nuevas características y mejoras, consolidando a Python como uno de los lenguajes de programación más importantes y ampliamente utilizados en la industria.